

LÆR AT KODE I PYTHON

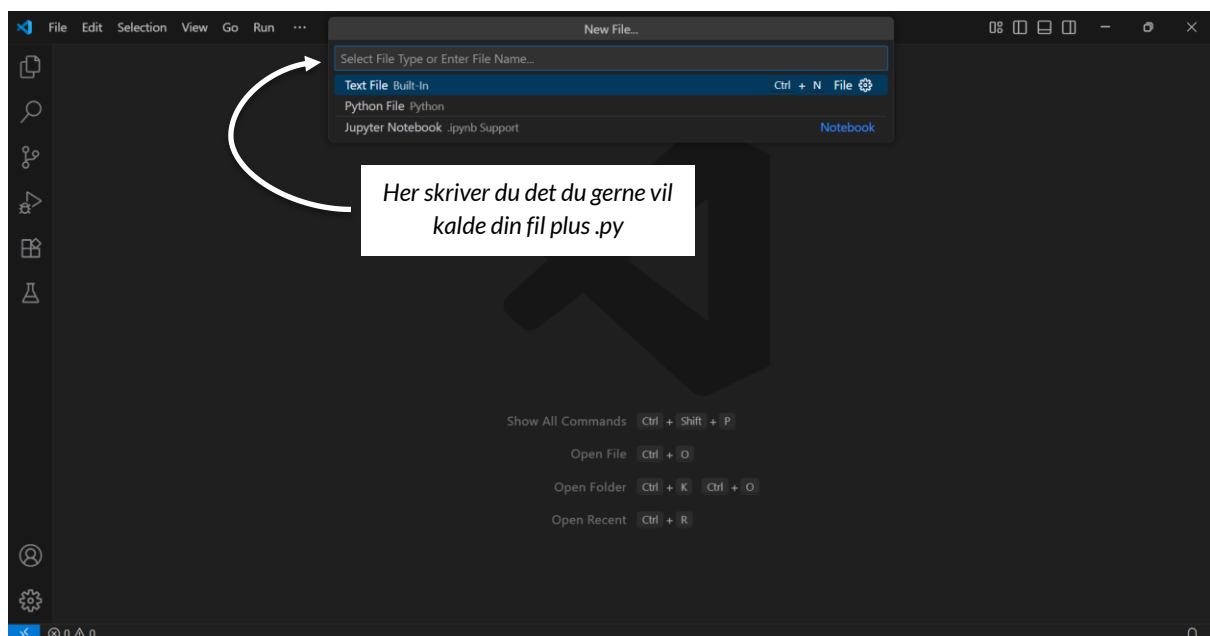
Fra Scratch til Python

Nu skal du gå fra blok programmering til linje kodning. Disse opgaver tager udgangspunkt i at du allerede kan finde ud af at kode i scratch, da der vil være mange sammenligninger for at give dig en let overgang fra Scratch til Python.

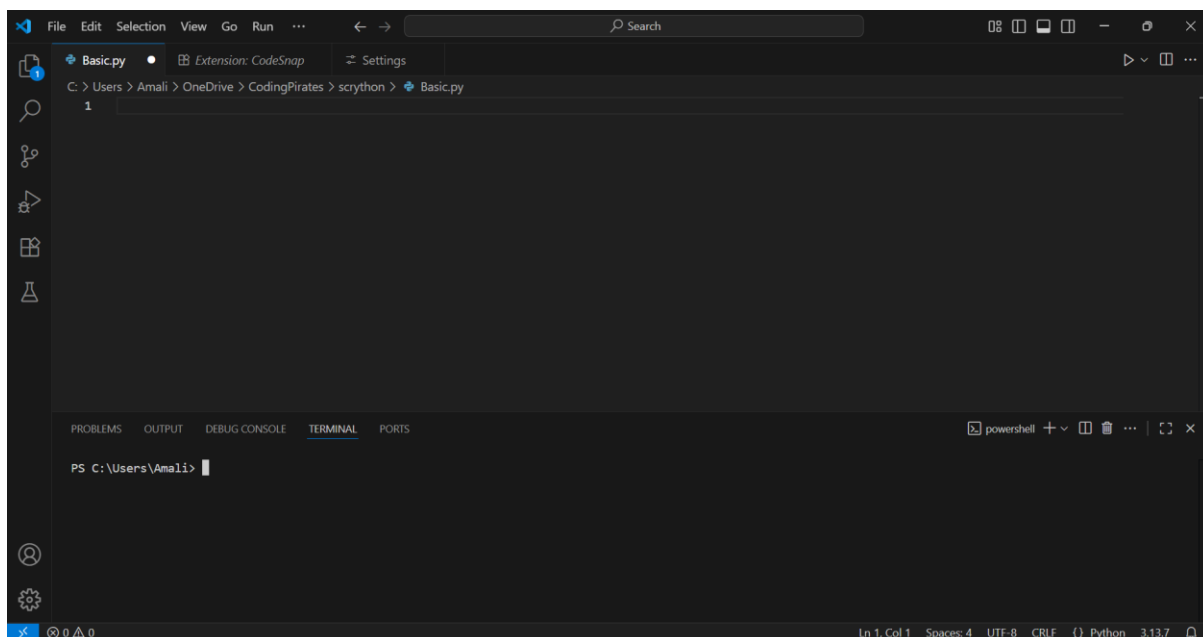
DET BASALE

Dette er alt det basale du skal forstå for at kode. Det er vigtigt at du forstår og gennemfører alle opgaverne.

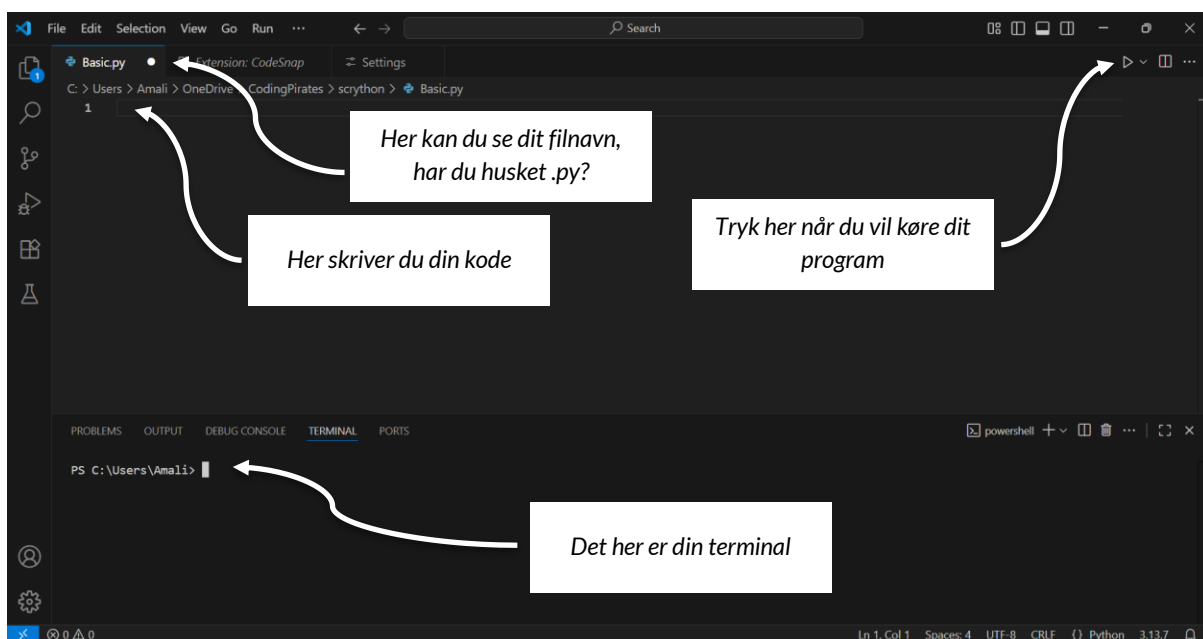
For at begynde skal du åbne Visual Studio Code, det er her inde du skal kode. For at begynde skal du først lave en ny fil. Dette gør du ved at trykke på "File" oppe i øverste venstre hjørne, derefter trykker du på "New File...". Din skræm burde se sådan her ud:



Det er meget vigtigt, at du skriver .py efter dit navn, ellers kommer det ikke til at virke. I eksemplet kalder vi filen for Basic.py og gemmer den på computeren. Nu burde din skærm se sådan ud:



Det her er de ting du skal ligge mærke til:



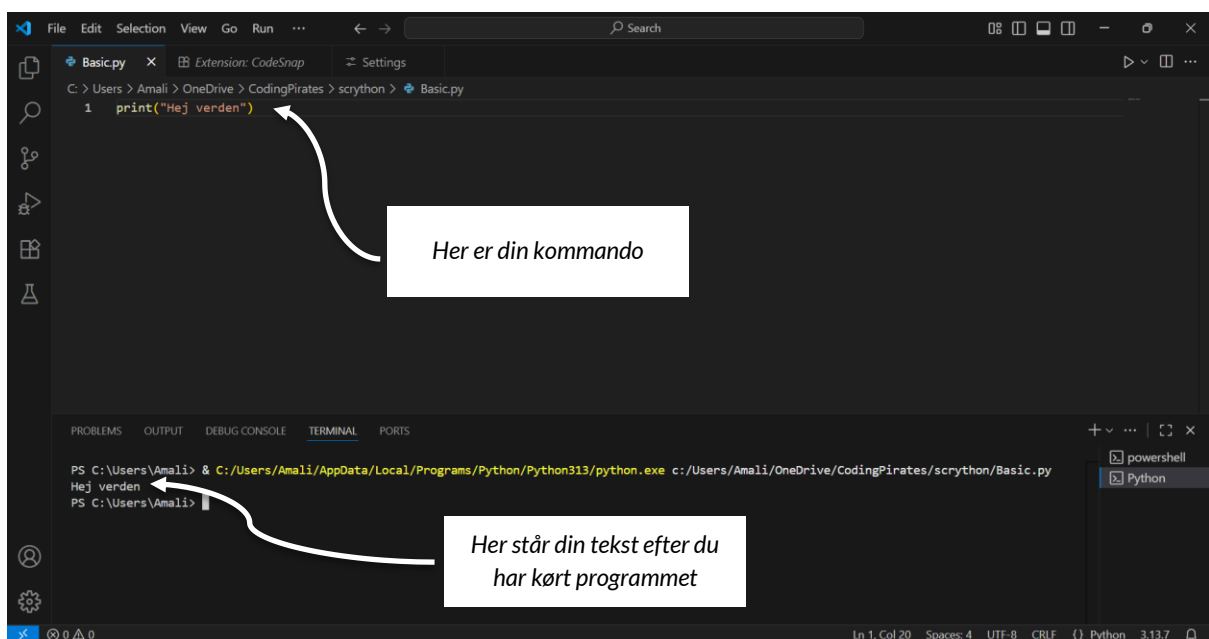
Godt klaret! Nu er du klar til at skrive din første linje kode.

1 Print til terminalen

Det første du skal lære er at skrive til din terminal. I Python skriver du til din terminal med kommandoen print. Det skrives sådan:

```
1 print("Hej verden")
```

Hvis du skriver et ord eller en sætning er vigtigt at du har "anførselstegn" om det du gerne vil printe i terminalen. Når du har skrevet din kommando, skal du gemme dit program ved at trykke "ctrl + s", herefter kan du køre dit program.



Opgave 1.1

Skriv noget til terminalen.

Opgave 1.2

Dit program bliver læst oppe fra ned.

Skriv to ting til terminalen og læg mærke til rækkefølgen det bliver printet i din terminal.

Opgave 1.3

Når du skriver et tal, behøver du ikke "anførselstegn", anførselstegn fortæller nemlig din computer, at du har skrevet tekst. Det vil sige at din computer tror "3" er et ord, men hvis du bare skriver 3, ved din computer at det er et tal.

Skriv et tal til terminalen.

Opgave 1.4

Prøv at skriv et regnestykke til terminalen, find helst på noget andet end eksemplerne. Prøv både at printe regnestykket med og uden anførselstegn.

```
1 print(1+2)
```

Hvad bliver der skrevet i terminalen, når der ikke er anførselstegn?
Skriv dit svar:

```
1 print("1+2")
```

Hvad bliver det skrevet i terminalen, hvis der er anførselstegn?
Skriv dit svar:

Opgave 1.5

Du kan printe både tekst og tal ved at opdele dem med komma. Det kunne se sådan ud:

```
1 print("Jeg er", 21, "år gammel")
```

Det betyder, at du kan bruge det du lærte fra sidste opgave og skrive:

```
1 print("1+2 =", 1+2)
```

Hvad tror du der kommer til at stå i terminalen?
Skriv dit svar:

Kør nu koden i filen "Opgave1-5.py" og se om du havde ret!

TIP!

Hvis du har brug for at skrive noter i din kode, skal du bruge # før du skriver din kommentar!

Så bliver det nemlig ignoreret af din computer.

```
1 # Det her er en kommentar!
```

2 Variabler

Variabler er måske noget du har hørt om i matematik, ellers kommer du til det senere hen. En variabel er et navn du giver til et tal. Dette kan også gøre i scratch med denne blok:



I Python gøres sådan:

```
1 variabel = 3
```

Opgave 2.1

Lav en variabel og print den til terminalen. Prøv både at printe variabelen med og uden anførselstegn.

```
1 print(variabel)
```

Hvad bliver der skrevet i terminalen, når der ikke er anførselstegn?
Skriv dit svar:

```
1 print("variabel")
```

Hvad bliver det skrevet i terminalen, hvis der er anførselstegn?
Skriv dit svar:

Kan du se at det er det minder meget om det der skete, da du lavede Opgave 1.4?

Opgave 2.2

Filen "Opgave2-2.py" indeholder to variabler, plus dem sammen. I terminalen skal der stå resultatet af dit regnestykke.

Skal du bruge anførselstegn? Hvorfor, hvorfor ikke?
Skriv dit svar:

Opgave 2.3

Du kan ændre værdien på din variabel, som du har lyst. Her bliver værdien af variabelen ændret fra at være 3 til at være 5:

```
1 variabel = 3
2 variabel = 5
```

I Opgave 1.2 lærte du, at dit program bliver læst oppe fra og ned, men kan du regne ud hvordan det påvirker dine variabler, når du printer dem?

Delopgave 1

```
1 variabel = 3
2 variabel = 5
3 print(variabel)
4 print(variabel)
```

Se på koden ovenover. Hvad kommer der til at stå i terminalen?
Skriv dit svar:

Linje 1:

Linje 2:

Kør nu koden i filen "Delopgave1.py" og se om du havde ret!

Delopgave 2

```
1 variabel = 3
2 print(variabel)
3 variabel = 5
4 print(variabel)
```

Se på koden ovenover. Hvad kommer der til at stå i terminalen?
Skriv dit svar:

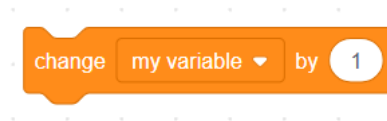
Linje 1:

Linje 2:

Kør nu koden i filen "Delopgave2.py" og se om du havde ret!

Opgave 2.4

I Scratch kan du ændre din variabel ved at tilføje til den. Det gør du med denne blok:



Det kan du også i Python ved at skrive:

```
1 variabel = variabel + 1
```

Det vil sige at hvis man skriver:

```
1 variabel = 10
2 variabel = variabel + 5
```

Går variablen fra at være 10 til at 15.

Lad os se, om du forstår det.


```
1 variabel = 5
2 print(variabel)
3
4 variabel = variabel + 5
5 print(variabel)
6
7 variabel = variabel + 10
8 print(variabel)
```

Kig på kode stykket ovenover. Hvad bliver der printet i terminalen?
Skriv dit svar:

1. print:

2. print:

3. print:

Kør nu koden i filen "Opgave2-4.py" og se om du havde ret!

Opgave 2.5

Nu skal du selv prøve. Lav en variabel og print den. Tilføj herefter 7 til din variabel og print den igen. Kør dit program og se om det virker.

Opgave 2.6

Man kan også tage et input, det gør du sådan her:

```
1 dit_input = input()
```

Med det kan man skrive kode:

```
1 print("Hvad er din ynglings farve?")
2 dit_input = input("Skriv her: ")
3 print(dit_input, "er en mega flot farve!")
```

Når programmet bliver kørt, bliver dette skrevet i terminalen:

```
Hvad er din ynglings farve?  
Skrive her: █
```

Ved den hvide boks kan man skrive noget:

```
Hvad er din ynglings farve?  
Skrive her: lilla█
```

Koden tager så dit input og bruger det i sætningen:

```
Hvad er din ynglings farve?  
Skrive her: lilla  
lilla er en mega flot farve!
```

Kør nu koden i filen "Opgave2-6.py" og se hvordan det virker.

Opgave 2.7

Lav noget kode hvor du selv skal bruge et input.

3 If-statement

If betyder hvis, derfor bliver et if-statement brugt, når noget skal ske hvis noget andet er sandt. Du kan for eksempel bruge et if-statement for at regne ud hvilke sko du skal tage på i dag. Hvis det regner, skal du tage gummistøvler på. Når du bruger et if-statement er du sikker på, at du ikke tager gummistøvler på, hvis det sner eller der er blå himmel.

I Scratch kender du if-statements som denne blokke:



Det kan man gøre i Python ved at skrive:

```
1 if():
```

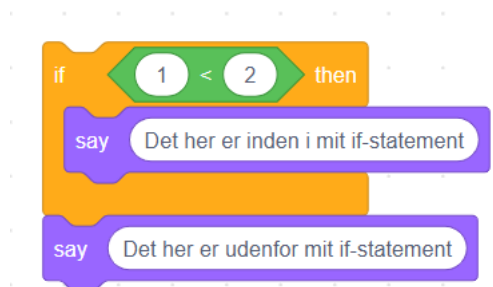
Her et eksempel på hvordan man kan udfylde blokken i Scratch:



Hvis du ville gøre det samme i Python ville det se sådan ud:

```
1 if(1<2):  
2     print("1 er mindre end 2")
```

Læg mærke til at linje 2 er skubbet længere ind end linje 1. Det betyder, at den linje ligger inden i dit if-statement, ligesom den lille blok ligger indenfor de gule bjælker i scratch blokken. Det er måske nemmere at se her:



```
1 if(1<2):
2     print("Det her er inden i mit if-statement")
3     print("Det her er udenfor mit if-statement")
```

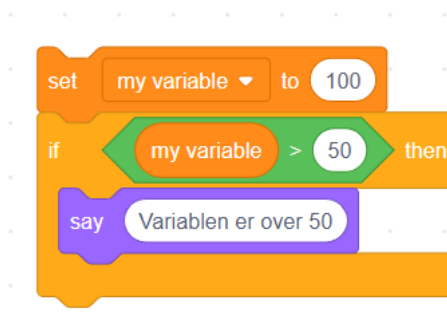
Disse to stykker kode gør det samme.

< og > kender du nok fra matematik, men her er nogle flere måder at sammenligne to tal i Python:

Betydning	Python
Det samme som	==
Mindre end	<
Større end	>
Mindre end eller det samme som	<=
Større end eller det samme som	>=
Ikke det samme som	!=

Opgave 3.1

Se på den kode blok og overvej hvad den gør:



Delopgave 1

Hvad gør kode blokken?

Skriv dit svar:

Delopgave 2

Skriv Python kode der gør det samme som kode blokken ovenover.

Opgave 3.2

Lav din egen variabel som tager imod et input, lav så et if-statement der printer din variabel, hvis den er større end 10.

Et input bliver altid set som en sætning. Du bliver derfor nødt til at konvertere dit input til et tal. Det gør du sådan her:

```
1 int(dit_input)
```

Int er en forkortelse for integer og betyder heltal på dansk.

Opgave 3.3

Det her står i filen "Opgave3-3.py":

```
1 import random
2 random_tal = random.randint(1, 10)
```

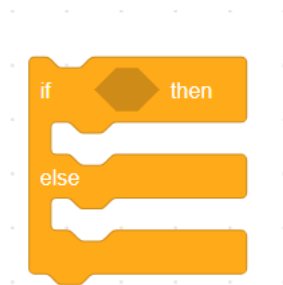
Den vil give dig et tilfældigt tal fra 1 til 10.

Lav nu et if-statement der printer det tilfældige tal, hvis det ikke er 5.

Opgave 3.4

Man kan også lave et if-else-statement, else betyder ellers. Dette bliver brugt, når der skal ske noget, hvis en ting er sandt, ellers skal der ske noget andet. Hvis det regner, skal du tage gummistøvler på, ellers skal du tage sneakers på.

Fra scratch kender du den som denne blok:



Det kan du også lave i Python ved at skrive:

```
1  if():
2      #indtæt kode
3  else:
4      #indsæt kode
```

Læg lige mærke til, at der bliver brug to kommentar!

Kig på dette stykke kode og husk det du har lært:

```
1  tal = 6
2
3  if(tal <= 5):
4      print(tal, "er det samme som eller under 5")
5  else:
6      print(tal, "er over 5")
```

Hvad bliver der skrevet i terminalen, når programmet bliver kørt?
Skriv dit svar:

Kør nu koden i filen "Opgave3-4.py" og se om du havde ret.

Opgave 3.5

Lav en variabel med et tilfældigt tal mellem 1 og 100. Print "VINDER!" hvis tallet er 47 ellers print "Bedre held næste gang".

Hint:

```
1  random.randint(1, 100)
```

Sådan laver du et tilfældigt tal mellem 1 og 100. Husk at lave tallet til en variabel ved at give den et navn.

Opgave 3.6

Man kan også lave en linje der hedder elif, det står for else if. Med den kan man tjekke for flere ting. Først tjekker du om det regner, hvis det regner, tager du gummistøvler på, hvis det ikke gør tjekker du om det sner, hvis det sner tager du vinterstøvler på, hvis det ikke gøre nogle af delene (ellers) tager du speakers på.

Det skrives således:

```
1  if():
2      #indsæt kode
3  elif():
4      #indsæt kode
5  else:
6      #indsæt kode
```

Computeren tjekker kun om den næste er sand, hvis den første ikke er sand. Det vil sige du tager gummistøvler på hvis det regner, selvom der er sne på jorden. Du tjekker kun om der er sne hvis det ikke regner. Husk det og kig på det her kode stykke:

```
1  tal = 5
2
3  if(tal == 5):
4      print("Tallet er 5")
5  elif(tal < 10):
6      print("Tallet er mindre end 10")
7  else:
8      print("Tallet er større end 10")
```

Hvad bliver der skrevet ud i terminalen? Og hvorfor tror du det?
Skriv dit svar:

Kør nu koden i filen "Opgave3-6.py" og se hvordan det virker.

Opgave 3.7

Lav en variabel med et tilfældigt tal mellem 1 og 10. Print "Midten" hvis tallet er 5, print "Lavt" hvis tallet er under 5, print "Højt" hvis tallet er over 5. Print altid din variabel.

Hvis du ikke kan huske, hvordan du laver et tilfældigt tal, kig tilbage på hintet i Opgave 3.5.

Opgave 3.8

Du kan også sammenligne ord eller sætningerne sådan her:

```
1  navn = input("Skriv dit navn: ")
2
3  if(navn == "Amalie"):
4      print("WOW! Amalie er et mega sejt navn")
5  elif(navn == "Kian"):
6      print("Kian er et godt navn")
7  else:
8      print(navn, "er det sejeste navn")
```

Kør nu koden i filen "Opgave3-8.py" og se hvordan det virker. Du må gerne prøve at ændre i koden eller lave dit eget stykke kode!

4 While-loop

While betyder imens, det vil sige at det kode, der befinder sig inde i et while-loop bliver ved med at gentage sig, imens noget andet er sandt. For eksempel er det jeres lærer der bestemmer, imens i er i skole.

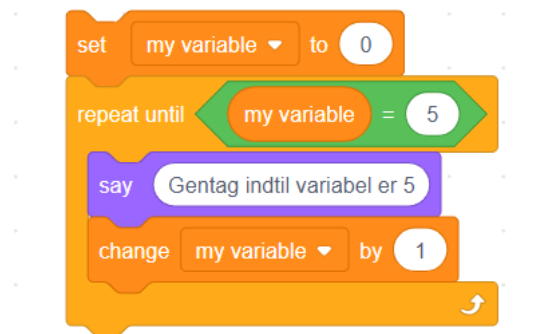
I scratch er det der kommer tættest på et while-loop denne blokke:



I Python skrives et while-loop sådan:

```
1 while():
```

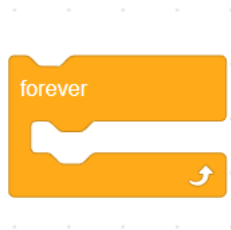
Men vær opmærksom på at de virker lidt forskelligt. Scratch blokken gentager indtil, mens Python linjen gentager imens. Se på disse to stykker kode:



```
1 variabel = 0
2
3 while(variabel < 5):
4     print("Gentag imens variabel er under 5")
5     variabel = variabel + 1
```

Kan du se, at de gør det samme? Begge loops stopper med at gentage deres indhold, når variabelen når til 5. Prøv gerne at køre koden i filen "Opgave4-0", prøv at ændre i koden, print evt. variabelen indtil du forstår hvad koden gør.

Du kan også lave denne blok ved hjælp af et while-loop:



Det ser sådan ud:

```
1 while(True):
```

Men vær meget opmærksom ikke at brug `while(True)` medmindre du ved hvad du laver. Din computer har ikke godt af det. Hvis du kommer til at gøre det skal du trykke "ctrl + c".

Opgave 4.1

Se på den kode blok og overvej hvad den gør:



Delopgave 1

Hvad gør kode blokken?

Hvis du ikke kan regne det ud, må du gerne lave det i Scratch, for at teste det.

Skriv dit svar:

Delopgave 2

Skriv Python kode der gør det samme som kode blokken ovenover.

Opgave 4.2

Skriv 5-tabellen i terminalen ved hjælp af while-loop.



5 For-loop

Et for-loop kan du bruge til tælle, det skrives sådan:

```
1 for i in range(start, slut, skridt_længde):
```

Hvis du gerne vil tælle 0-10 skriver du 0 ved start, og 11 ved slut.:

```
1 for i in range(0, 11, 1):  
2     print(i)
```

Grunden til at du vil skrive 11 i stedet for 10 ved slut, er fordi den tæller op til slut, ikke til og med. Skridtlængde er det der bliver plusset med hver gang. Hvis den er 1 vil der skrives 0, 1, 2 osv. Hvis skridtlængden er 2 skrives der 0, 2, 4 osv.

Opgave 5.1

I filen "Opgave5-1.py" kan du finde dette stykke kode:

```
1 start = 0  
2 slut = 10  
3 skridtlængde = 1  
4  
5 for i in range(start, slut, skridtlængde):  
6     print(i)
```

Kør koden og se hvad den gør. Prøv også at ændre på de tre variabler og se hvordan det ændre på udkommet af koden.

Opgave 5.2

Print 3-tabellen i terminalen ved hjælp af for-loop.

Opgave 5.3

Tæl til 100 og skriv når dit yndlingstal bliver printet i terminalen. Du skal bruge et for-loop og et if-statement.

6 Lister

Lister er noget du kan bruge til at holde flere data, ligesom når du laver en ønskeliste.

Sådan laver du en liste i Python:

```
1 liste = ["ting 0", "ting 1", "ting 2"]
```

Dette er en liste med 3 ting, men læg mærke til at første ting i listen hedder ting 0, det er fordi man tæller fra 0, når man programmerer. Det vil sige at selvom der er 3 ting i listen har de numrene 0, 1 og 2.

For at vælge noget fra listen skrives:

```
1 liste[tal]
```

Det vil sige at hvis man skriver:

```
1 print(liste[0])
```

Og man kører sin kode vil der blive skrevet ting 0 i konsollen. Hvis du åbner filen "Opgave6-0.py" kan du køre koden, leg med koden indtil du forstår hvad der sker!

Opgave 6.1

Lave en liste af tal og print et af elementerne i listen.

Opgave 6.2

Man kan printe alle elementer i en liste ved at skrive:

```
1 for i in liste_navn:  
2     print(i)
```

"i" er et element i en liste.

Åben filen "Opgave6-2.py" og se hvad der sker, når du kører programmet.

Gå tilbage til den kode du skrev i opgave 6.1 og print hele din liste.

Opgave 6.3

Man kan tilføje ting til en liste ved hjælp af "append" det gøres sådan:

```
1 liste_navn.append(element)
```

Her laves en tom liste, og der tilføjes tre tal til listen:

```
1 liste = []
2
3 liste.append(1)
4 liste.append(2)
5 liste.append(3)
6
7 for i in liste:
8     print(i)
```

Kør programmet og tilføj herefter endnu et element til listen.

Opgave 6.4

Tilføj 6 tabellen til en liste, du skal bruge enten et for-loop eller et while-loop. Du kan bruge begge, vælg den du synes er lettest.

Print hele din liste for at vise at du har udført opgaven korrekt.

Opgave 6.5

Man kan fjerne noget fra en liste ved at skrive remove(element):

```
1 liste_navn.remove(element)
```

Det fjerner det element du specificerer.

Se på koden:

```
1 liste = ["Ged", "ko", "Får"]
2
3 print("Før ko bliver fjernet:")
4 for i in liste:
5     print("-", i)
6
7 liste.remove("Ko")
```

Hvad tror du der kommer til at stå, hvis man printer listen igen?
Skriv dit svar:

Kør nu koden i filen "Opgave6-5.py" og se om du havde ret.

Opgave 6.6

Filen "Opgave6-6.py" indeholder denne liste:

```
1 liste = [1, 2, 3, 4, 5, 1]
```

Fjern et 1-tal fra listen og print listen.

Hvad er der sket med listen? Hvilken betydning havde det at der var to 1-taller? Blev begge fjernet eller var det kun et af dem?

Skriv dit svar:

Opgave 6.7

Man kan også fjerne et bestemt nummer i listen. Hvis du tænker helt tilbage til det første du læste, og kigger på dette stykke kode:

```
1  liste = [1, 2, 3, 4, 5]
2
3  liste.remove(liste[3])
```

Hvilket tal bliver fjernet? Du må gerne gå tilbage og læse den første del af afsnittet igen. Skriv dit svar:

Kør nu koden i filen "Opgave6-7.py" og se om du havde ret.

Opgave 6.8

Filen "Opgave6-8.py" indeholder denne liste:

```
1  små_dyr = ["Sommerfugl", "Bænkebidder", "Bille", "Myg", "Edderkop"]
```

Tilføj endnu et dyr til listen og fjern bænkebidder. Du skal gøre det ved at skrive små_dyr[tal].

Print listen og se om du gjorde det.

7 Brug det du har lært!

Tillykke! Nu kan du grundprincipperne for at kode. Her er nogle lidt svære opgaver, hvor du skal bruge alt det, du har lært!

Opgave 7.1 – Interview

Stil nogle spørgsmål i terminalen, spørgsmålene skal kunne besvares.

Lav en opsummering på alt det du ved om personen nu.

Opgave 7.2 – Sten saks papir

Få din terminal til printe enten sten, saks eller papir. Det skal selvfølgelig være tilfældigt hvad den vælger.

Opgave 7.3 – Gæt tallet

Generer et tilfældigt tal og lad spilleren gætte det. Fortæl spilleren hvad det rigtige tal var og om de har vundet eller ej.

Opgave 7.4 – Lav en indkøbsliste

Lav en indkøbsliste.

Man skal kunne blive ved med at tilføje ting til indkøbslisten.

Hvis der skrives "Vis", skal man kunne se hele listen.

Hint:

```
1 while(True):
```

Delopgave 1

Gør så man kan fjerne en ting på listen efter man skriver "Slet".

Delopgave 2

Sørg for at programmet ikke går i stykker, hvis man prøver at slette noget der ikke er på listen.

Hint:

```
1 liste_navn.__contains__(element)
```



Den har du ikke set før, men prøv at leg med den, lær hvordan den virker. Husk print() er din bedste ven!

Delopgave 3

Gør så man kan skrive "Slet alt". Det har du ikke lært hvordan man gør, men prøv at finde ud af det.